1 Formulas

A formula is one expression that produces a value. It can combine literals, variables, operators, function calls, local bindings, and comprehensions. Types are checked during evaluation. Lists and dictionaries can contain mixed types, and collection operations return new values.

```
let price = 12, quantity = 3 for price * quantity
[1, 2, 3].map(fn value -> value * 2)
'yes' if 4 > 2 else 'no'
```

These expressions produce `36`, `[2, 4, 6]`, and `'yes'` respectively. Each line in an example is an independent formula. Method and function signatures below use square brackets around optional arguments; those brackets are documentation notation, not part of a call.

Contents

1 Formulas 1
1.1 Source and names 1
1.2 Datatypes and literals 2
1.2.1 Numbers 2
1.2.2 Strings 2
1.2.3 Lists and dictionaries 3
1.2.4 Truthiness and conversion 3
1.3 Operators 3
1.3.1 Arithmetic 3
1.3.2 Comparisons and membership 4
1.3.3 Logic, fallback, and conditionals 4
1.3.4 Bitwise operations 5
1.3.5 Precedence and associativity 5
1.4 Formal grammar 5
1.4.1 Expressions and precedence 6
1.4.2 Values and postfix operations 6
1.4.3 Bindings, functions, and comprehensions 7
1.5 Indexing, slicing, and members 7
1.6 Local bindings and functions 8
1.6.1 Temporary bindings 8
1.6.2 Function expressions and calls 8
1.6.3 Comprehensions 9
1.7 Default global functions 9
1.7.1 Values and numbers 9
1.7.2 Collections and callbacks 10
1.7.3 Evaluation and errors 11
1.8 Type methods 11
1.8.1 Integer and float methods 11
1.8.2 String methods 12
1.8.3 List methods 13
1.8.3.1 Values, ordering, and selection 13
1.8.3.2 Callbacks 14
1.8.4 Dictionary methods 14
1.9 Text formatting and embedded formulas 15

1.1 Source and names

Whitespace, including newlines, separates tokens and otherwise has no effect. Parentheses group an expression. There are no statement separators, comments, tuple literals, or general assignment statements. `//` is floor division; `#` begins a hexadecimal integer.

Names are case-sensitive. Ordinary identifiers start with a letter or underscore and continue with letters, digits, or underscores; Unicode letters and digits are accepted. A variable reference may have a leading `$`: `score` and `$score` refer to the same binding. Variables come from the calling environment, local bindings, function parameters, or the default globals. An unknown name is an error.

The following words cannot be used as local binding names or function parameters:

```
let for in if else and or xor not fn
True False None true false null
```

Function parameters also cannot be named `args`. Builtin names such as `sum` and `range` are ordinary global bindings and can be shadowed. Dictionary property names may use keywords: `{'let': 1}.let` is valid.

A complete formula may be written as `${expression}` for explicit evaluation with `eval`. Text substitution is described at the end of this reference.

1.2 Datatypes and literals

Type returned by `type(value)`	Values and literal syntax
`"null"`	`null` or `None`: absence of a value.
`"bool"`	`true` / `True` and `false` / `False`.
`"int"`	Signed 64-bit integers, such as `42`, `-7`, `0xff`, `0b101`, `0o17`, and `#ff`.
`"float"`	64-bit floating-point numbers, such as `3.14` and `-0.5`.
`"str"`	Unicode strings in single or double quotes.
`"list"`	Ordered values: `[1, 'two', null]`; empty list: `[]`.
`"dict"`	String-keyed entries: `{'name': 'Ada', 'level': 3}`; empty dictionary: `{}`.
`"func"`	Callable values: builtin functions and `fn x -> x + 1`.

1.2.1 Numbers

Integer prefixes `0x`, `0b`, and `0o` also accept uppercase `X`, `B`, and `O`. Hexadecimal digits are case-insensitive. Radix literals must fit in a positive signed 64-bit integer before a unary minus is applied. There are no digit separators.

A float literal needs digits on both sides of its decimal point: use `0.5` and `1.0`, not `.5` or `1.`. Scientific notation is accepted by string conversion, such as `float('1e3')`, but not by numeric literal syntax. Negative numbers use the unary `-` operator.

`nan()` returns a floating-point NaN. `inf()` returns positive infinity; `-inf()` gives negative infinity. Use `.isnan()` and `.isinf()` to test these values. `nan() == nan()` is false. Ordered comparisons of NaN against a float or boolean raise an error; against an integer, NaN follows the total numeric ordering described under `order`. Infinities compare numerically. NaN and infinities are truthy.

Integer-to-float conversion can lose precision for large integers. Floating-point arithmetic has normal rounding limitations. Keep integer calculations within the signed 64-bit range; overflow handling varies between operations.

1.2.2 Strings

Both quote styles produce the same type. Ordinary strings recognize `\n`, `\r`, `\t`, `\\`, `\'`, and `\"`. Unicode codepoint escape are also supported with `\uXXXX` syntax. For other escapes, the backslash is discarded: `'\q'` contains `q`. A string may contain a literal newline.

```
'hello'
"it's ready"
'line one\nline two'
```

String literals are values; `$name` or `${...}` inside a string is not automatically evaluated. A quoted string consisting exactly of `${...}` has special delimiter handling: the entire contents are preserved, including inner quotes and backslashes. For example, `'${'ready'}'` is a string containing `${'ready'}`. See text formatting for explicit evaluation.

String length, indexing, slicing, `indexof`, and iteration count Unicode scalar values, rather than bytes or visual grapheme clusters.

1.2.3 Lists and dictionaries

List items, dictionary keys, and dictionary values are expressions. Dictionary keys must evaluate to strings; a bare key name is a variable reference, not an implicit string. Entries retain insertion order. Repeating a key replaces its value without moving its position. Lists and dictionaries allow a trailing comma.

```
[1, 2 + 3, 'four',]
let key = 'score' for {key: 10, 'double': 20}
{'x': 1, 'x': 2}
```

Lists compare element by element for equality. Dictionaries compare their keys and values, independently of insertion order. Collection elements use the same equality rules as standalone values.

1.2.4 Truthiness and conversion

The false values are `null`, `false`, numeric zero (including `-0.0`), `''`, `[]`, and `{}`. All other values are true, including functions. `bool(value)` returns this truthiness.

`int(value)` accepts integers, floats, booleans, and decimal integer strings. Floats truncate toward zero; out-of-range floats saturate at the integer limits, and NaN converts to zero. `true` converts to `1`, and `false` to `0`. A string such as `'3.5'` or `'0xff'` is not an integer conversion input.

`float(value)` accepts integers, floats, booleans, and floating-point strings, including exponent notation. Numeric string conversions do not strip surrounding whitespace. `null`, lists, dictionaries, and functions cannot be converted to numbers.

`str(value)` returns a display representation. Strings include surrounding double quotes, booleans use lowercase names, builtin functions appear as `<builtin name>`, and user-defined functions display as the closure definition. This representation is for display, not a guaranteed round-trip serialization. Text formatting inserts string contents without adding those quotes.

```
str(fn x -> x * 2)
"<fn x -> ((x) * (2))>"
```

1.3 Operators

1.3.1 Arithmetic

Expression	Meaning
`+x`, `-x`	Numeric identity and negation; accept integers or floats.
`a + b`	Numeric addition; also concatenates two strings or two lists, or merges two dictionaries with right-hand values winning.
`a - b`	Subtraction. Two integers produce an integer; otherwise operands are converted with `float` semantics.
`a * b`	Numeric multiplication; also repeats a string by a nonnegative integer, in either operand order.
`a / b`	Division, always returning a float. Operands use `float` conversion semantics. A zero divisor is an error.
`a // b`	Division rounded down toward negative infinity. Two integers produce an integer; otherwise the result is a float.
`a % b`	Remainder; see the type-dependent sign rules below.
`pow(a, b)`	Exponentiation.

For `+` and `*`, mixing an integer with a float produces a float. Two booleans, or a boolean paired with an integer, instead produce a boolean: `+` performs truthy OR and `*` performs truthy AND. A boolean paired with a float is unsupported by these two operators. Strings are not implicitly converted for addition or multiplication.

`//` and `%` accept integer/float operands, not booleans or numeric strings. Integer `//` computes an exact integer quotient rounded down, including for negative divisors, and rejects a result outside the signed 64-bit range. Integer `%`,

and `%` with exactly one float, use a remainder with the dividend's sign. With two floats, `%` computes `a - floor(a / b) * b`, giving the divisor's sign for ordinary finite inputs. Both reject a zero divisor.

```
-7 // 3
-7 % 3
-7.0 % 3.0
'ab' * 3
{'a': 1} + {'a': 2, 'b': 3}
```

The numeric results are `-3`, `-1`, and `2.0`. String repetition gives `'ababab'`; dictionary addition gives `{'a': 2, 'b': 3}`.

`pow` returns an integer for an integer base and a nonnegative integer exponent, with checked overflow. Other supported inputs produce a float. Zero to a negative power or complex results are errors.

1.3.2 Comparisons and membership

==, `!=`, `<`, `<=`, `>`, and `>=` compare values. Integers, floats, and booleans can compare numerically, with booleans represented by `0` and `1`. Mixed integer/float comparisons preserve integer precision instead of rounding large integers to floats. Signed-zero comparisons are type-dependent: `0.0 == -0.0` is true, but `0 == -0.0` is false and `0 > -0.0` is true. Strings compare lexicographically, and lists use lexicographic element comparison. `null` compares equal to itself. Dictionaries and functions do not support ordered comparisons; incompatible ordered operands raise an error.

=== tests strict equality, considering both type and value, recursively for lists and dictionaries. `!==` tests strict inequality. Strict equality for function values compares their identity, not just their behavior. Float strict equality still treats NaN as unequal to itself and the two signs of zero as equal.

Equality of unrelated types is false. Ordinary function equality compares builtin names or a user function's definition and captured state; strict function equality compares identity.

Comparisons chain. `a < b <= c` checks each adjacent pair, evaluates the middle expression once, and stops at the first false comparison.

`item in container` and `item not in container` test:

- A list: equality with an element.
- A string: substring presence; the item must be a string.
- A dictionary: key presence; the item must be a string.

```
1 == true
1 < 2 <= 3
'cat' in 'catalogue'
'level' in {'level': 3}
```

All four expressions return `true`.

1.3.3 Logic, fallback, and conditionals

Expression	Result and evaluation
`a and b`, `a && b`	Return `a` if it is falsey; otherwise evaluate and return `b`.
`a or b`, `a || b`	Return `a` if it is truthy; otherwise evaluate and return `b`.
`a xor b`	Evaluate both operands and return whether their truthiness differs. Always returns a boolean.
`not x`, `!x`	Boolean negation of truthiness. These spellings have different precedence.
`a ?? b`	Return `a` unless it is `null`; only then evaluate `b`.
`a if condition else b`	Evaluate the condition, then only the selected branch. Return that branch's value.

```
0 or 10
0 ?? 10
false and (1 / 0)
'positive' if 3 > 0 else 'other'
```

The results are `10`, `0`, `false`, and `'positive'`. Null fallback does not suppress errors while evaluating its left operand: an unknown variable or missing member still fails. Use the collection's `tryget` method for an optional lookup.

1.3.4 Bitwise operations

Binary `&`, `|`, and `^` mean bitwise AND, OR, and XOR. They accept integers and booleans, converting booleans to `0` or `1`. Unary `~` complements an integer's 64 bits; on a boolean it first converts to `0` or `1` and returns an integer.

Shifts accept integer or boolean operands and require a nonnegative shift count after boolean conversion. `<<` shifts left, `>>` shifts right with sign extension, and `>>>` shifts right with zero fill. Bits shifted out are discarded. For counts of 64 or more, left and logical right shifts return zero; arithmetic right shifts return `-1` for negative inputs and zero otherwise.

```
0b1100 & 0b1010
1 << 4
-8 >> 1
```

The results are `8`, `16`, and `-4`.

1.3.5 Precedence and associativity

The table runs from strongest binding to weakest. Parentheses override it.

Syntax	Associativity
`f(...)`, `x.name`, `x[i]`, `x[a:b:c]`	Postfix operations chain left to right.
`+x`, `-x`, `!x`, `~x`	Unary, right to left.
`*`, `/`, `//`, `%`	Left to right.
`+`, `-`	Left to right.
`<<`, `>>`, `>>>`	Left to right.
`&`	Left to right.
`^`	Left to right.
`|`	Left to right.
`==`, `!=`, `===`, `!==`, `<`, `<=`, `>`, `>=`, `in`, `not in`	Chained comparisons.
`not`	Unary, right to left.
`and`, `&&`	Left to right.
`xor`	Left to right.
`or`, `||`	Left to right.
`??`	Right to left.
`a if condition else b`	Right-associative through the else branch.

Thus `not a == b` means `not (a == b)`, while `!a == b` means `(!a) == b`. `let` and `fn` may appear wherever an operand is accepted, but their bodies consume a full expression: `2 * let x = 3 for x + 4` means `2 * (let x = 3 for x + 4)`.

1.4 Formal grammar

The following EBNF describes expressions after tokenization. Quoted text denotes a terminal; commas join successive items, `|` separates alternatives, brackets denote an optional item, and braces denote repetition. Parentheses group grammar items. `end_of_input` requires the entire expression to be consumed. Whitespace may separate tokens. The `${...}` delimiters belong to explicit evaluation and text formatting, not to the expression grammar.

1.4.1 Expressions and precedence

```
formula       = expression, end_of_input ;
expression    = conditional ;
conditional   = coalescing,
                ["if", coalescing, "else", expression] ;
coalescing    = disjunction, ["??", coalescing] ;
disjunction   = exclusive_or, {("or" | "||"), exclusive_or} ;
exclusive_or  = conjunction, {"xor", conjunction} ;
conjunction   = negation, {("and" | "&&"), negation} ;
negation      = "not", negation | comparison ;
comparison    = bit_or, {comparison_op, bit_or} ;
comparison_op = "==" | "!=" | "===" | "!=="
              | "<" | "<=" | ">" | ">=" | "in" | "not", "in" ;
bit_or        = bit_xor, {"|", bit_xor} ;
bit_xor       = bit_and, {"^", bit_and} ;
bit_and       = shift, {"&", shift} ;
shift         = sum, {("<<" | ">>" | ">>>"), sum} ;
sum           = product, {("+" | "-"), product} ;
product       = unary, {("*" | "/" | "//" | "%"), unary} ;
unary         = ("+" | "-" | "!" | "~"), unary | postfix ;
postfix       = primary, {suffix} ;
```

Repetitions of binary operators associate left to right, except that comparisons form a chain. The recursive right operands of `??` and the conditional express their right associativity. At every level, an available operator is consumed before returning to the enclosing expression. Likewise, `let` and `fn` bodies extend through the full expression that follows them; parenthesize the whole construct to apply an operator or postfix operation to its result.

1.4.2 Values and postfix operations

```
primary    = scalar | identifier | dice
           | "(", expression, ")" | list | dictionary
           | let_expression | function_expression ;
scalar     = integer | float | string
           | "true" | "True" | "false" | "False"
           | "null" | "None" ;
suffix     = "(", [arguments], ")"
           | ".", (identifier | integer)
           | "[", expression, "]"
           | "[", [expression], ":", [expression],
             [":", [expression]], "]" ;
arguments  = expression, {",", expression} ;
list       = "[", "]"
           | "[", expression, {",", expression}, [","], "]"
           | "[", expression, comprehension, "]" ;
dictionary = "{", "}"
           | "{", entry, {",", entry}, [","], "}"
           | "{", entry, comprehension, "}" ;
entry      = expression, ":", expression ;
```

`identifier`, `integer`, `float`, and `string` denote the lexical forms described under source and literals. Numeric tokens are unsigned in spelling; signs are unary operators. `dice` denotes decimal digits, lowercase `d`, and decimal digits with no intervening whitespace, as in `2d6`; it is shorthand for a call to `dice`.

An optional leading `$` is removed from identifiers before recognizing names or keywords. In expression positions, the literal words and `let`, `fn`, and unary `not` take precedence over variable references; after a dot, keywords are accepted as property names. A dotted integer must be nonnegative. Adjacent characters that form a single token stay together: use `0.5` for a float and `(1).abs()` to make an integer receiver unambiguous.

1.4.3 Bindings, functions, and comprehensions

```
let_expression      = "let", binding, {",", binding},
                      "for", expression ;
binding             = pattern, "=", expression ;
function_expression = "fn", [parameters], "->", expression ;
parameters          = pattern, {",", pattern} ;
pattern             = binding_name | list_pattern ;
list_pattern        = "[", [pattern_items], "]" ;
pattern_items       = pattern_item, {",", pattern_item}, [","] ;
pattern_item        = pattern | rest_pattern ;
rest_pattern        = "*", binding_name ;
comprehension       = "for", targets, "in", coalescing,
                      ["if", coalescing] ;
targets             = pattern_item, {",", pattern_item} ;
```

The grammar is subject to these binding constraints:

- `binding_name` is an identifier other than a reserved word listed under source and names. `_` is allowed and discards its value. Within one `let`, parameter list, or comprehension target list, names must be unique across all nested patterns, except for `_`.
- Function parameter patterns also exclude `args`, including inside nested lists and rest captures.
- Each list-pattern level, and the outer comma-separated comprehension target list, permits at most one rest capture. A rest capture cannot be the sole unbracketed comprehension target; use `[*rest]` instead. A bare rest capture is not a `let` binding or function parameter.

List and dictionary literals and list patterns allow trailing commas. Calls, function parameter lists, `let` binding lists, and comprehension target lists do not. A comprehension has only one `for` and one optional filter; its source and filter use `coalescing`, so a conditional there must be parenthesized. Value-type and destructuring-length requirements are checked during evaluation, as described in the corresponding sections.

1.5 Indexing, slicing, and members

Lists and strings use zero-based integer indexes. Negative indexes count from the end: `-1` is the last element. An out-of-range index is an error. Both also accept dotted nonnegative indexes, such as `[10, 20].0` or `'abc'.1`.

Dictionary lookup accepts a string key in brackets or a property name after a dot (if the name is a valid identifier). Use brackets for computed keys or names containing punctuation.

```
[10, 20, 30][-1]
'café'[3]
{'name': 'Ada'}.name
```

These return `30`, `'é'`, and `'Ada'`.

Lists and strings support `value[start:end:step]`. The end is exclusive. Any field can be omitted or set to `null`; other bounds must be integers. The default step is `1`, and zero is invalid. Negative bounds count from the end; out-of-range bounds are clamped. With a negative step, omitted start/end bounds traverse from the last element toward the beginning.

```
[0, 1, 2, 3, 4][1:4]
[0, 1, 2, 3, 4][::2]
'abcde'[::-1]
[0, 1, 2, 3, 4][-3:]
```

The results are `[1, 2, 3]`, `[0, 2, 4]`, `'edcba'`, and `[2, 3, 4]`. An omitted reverse end differs from an explicit `-1`: the latter refers to the last element, not the position before the beginning.

A member lookup first checks stored data, then the type's method registry. A dictionary key such as `'len'` therefore takes precedence over the method of that name. A missing data key may resolve to a registered method; use `.tryget(key)` when only dictionary data should be considered.

Direct registered-method calls supply the receiver automatically: `values.map(f)` and `values['map'](f)` both do so. A stored function-valued dictionary entry receives only the arguments written in the call. An extracted method is an ordinary, unbound function and needs its receiver explicitly:

```
let length = [].len for length([10, 20])
{'double': fn x -> x * 2}.double(5)
```

These return `2` and `10`.

1.6 Local bindings and functions

1.6.1 Temporary bindings

Use `let pattern = value, ... for body`. Bindings are evaluated sequentially; each can refer to earlier bindings. They shadow outer names only inside the expression. The expression's result is the body's result.

```
let x = 2, y = x + 3 for x * y
let [x, [y, _]] = [2, [3, 99]] for x + y
```

The results are `10` and `5`. A binding pattern is a name or a nested list of patterns. Without a rest capture, a list pattern requires a list of exactly the corresponding length at every level. _ discards a value and can be repeated. Other names must be unique across one `let`'s binding patterns. Empty patterns and trailing commas inside list patterns are allowed.

Discarding a value does not skip its evaluation: `let _ = error('failed') for 0` still raises an error. Surrounding list patterns still enforce their type and length requirements, and discarded function parameters still count toward the minimum argument count and remain in `args`.

_ creates no binding and does not shadow an existing variable named _. Outside a pattern, _ is an ordinary variable reference. Thus `let _ = 8 for _` returns an existing outer _, or fails if none is defined. `$_` has the same meaning as _, including in patterns; names such as `_value` are ordinary names.

Inside a list pattern, `*name` captures the remaining elements as a list, possibly empty. It may appear at the beginning, middle, or end: elements before it match the prefix, and elements after it match the suffix. Each list level allows at most one rest capture. The input must contain enough elements for all non-rest patterns. Use `*_` to discard the remaining elements. The capture must be a name, not another pattern.

```
let [head, *tail] = [1, 2, 3] for [head, tail]
let [first, *middle, last] = [1, 2] for middle
let [[head, *tail], *rows] = [[1, 2], [3], [4]] for [tail, rows]
```

These return `[1, [2, 3]]`, `[]`, and `[[2], [[3], [4]]]`. The same list patterns work in function parameters, for example `(fn [head, *tail] -> tail)([1, 2, 3])`. Rest captures destructure list values; they do not change a function's positional argument count.

1.6.2 Function expressions and calls

Use `fn parameter, ... -> body`, or `fn -> body` for no declared parameters. Parameters support the same list patterns and discard marker as `let`. Names must be unique across parameters. The body is one expression; it can contain another `let`, function, or conditional.

```
(fn x, y -> x + y)(2, 3)
(fn [x, y] -> x * y)([4, 5])
let offset = 10, add = fn x -> x + offset for add(2)
(fn -> args.len())(10, 20, 30)
```

The results are `5`, `20`, `12`, and `3`. Calls use positional arguments. Function calls and parameter lists do not allow a trailing comma. Parenthesize a function expression before calling it immediately.

User functions require at least as many arguments as declared parameters. Extra arguments are accepted, and `args` contains the complete argument list. This lets a callback ignore extra context such as an element index. Builtins enforce their own argument counts; passing a builtin directly as a callback can fail if it accepts fewer arguments than the callback receives.

Functions capture referenced local values when they are created. These captures take precedence over the calling environment, regardless of the variable's name. Global bindings remain dynamic and are resolved when the function is called. Parameters and `args` take priority over captures; names unresolved at creation may fall back to the caller's environment.

Arguments are evaluated eagerly, left to right, before resolving and calling the callee or method receiver. Merely creating an `fn` value does not evaluate its body. Evaluation errors propagate to the caller.

Builtins and user-defined functions share a maximum active call depth, which defaults to 32 but can be configured. This includes nested calls through `eval`, `apply`, and callbacks. Exceeding this limit produces a `RecursionLimit` error that `try` can handle. Each returning call releases its depth slot, including on error; sequential callbacks do not accumulate depth.

1.6.3 Comprehensions

A list comprehension is `[expression for target in source if condition]`. A dictionary comprehension is `{key: value for target in source if condition}`. The filter is optional and tested before producing an output item. Source values are evaluated once; loop names are local to each iteration. Dictionary output keys must be strings; later duplicate keys replace earlier values.

```
[x * x for x in range(5) if x % 2 == 0]
{key: value * 2 for key, value in {'a': 1, 'b': 2}}
[a + b for a, b in [[1, 2], [3, 4]]]
```

These produce `[0, 4, 16]`, `{'a': 2, 'b': 4}`, and `[3, 7]`.

Source	Loop targets
List	One pattern receives each item. Multiple comma-separated patterns unpack each item as a list, with the same length and rest-capture rules as a bracketed list pattern.
Dictionary	One pattern receives each key; two patterns receive key and value, in insertion order. A value pattern can destructure a stored list.
String	One pattern receives each Unicode scalar value as a string.

Comprehension targets support nested list patterns and rest captures, with _ and *_ available to discard components. Names must be unique across one comprehension's target patterns, except for _. A rest capture must be inside brackets or part of multiple comma-separated targets; use `[*all]` to capture an entire list. Shape mismatches raise an error, even if a filter would exclude that item. Bound names are available to the filter, output expressions, and functions created within the comprehension, and do not leak into the outer scope.

```
[[head, tail] for [head, *tail] in [[1, 2, 3], [4]]]
[middle for first, *middle, last in [[1, 2, 3, 4], [5, 6]]]
{key: tail for key, [head, *tail] in {'a': [1, 2], 'b': [3]}}
```

These return `[[1, [2, 3]], [4, []]]`, `[[2, 3], []]`, and `{'a': [2], 'b': []}`. For a list source, `for [a, b] in rows` and `for a, b in rows` both unpack each row. Dictionary iteration still distinguishes one target (the key) from two targets (key and value).

A comprehension has one `for` clause and at most one filter. Nest comprehensions for nested iteration. Parenthesize a conditional expression used as the source or filter to distinguish it from the comprehension's `if`.

1.7 Default global functions

The following functions are available as standard global bindings. Their names can be shadowed like other variable names.

1.7.1 Values and numbers

Call	Behavior

`type(value)`	Return the type name listed under datatypes.
`str(value)`, `int(value)`, `float(value)`, `bool(value)`	Convert as described under truthiness and conversion.
`exists(name)`	Return whether a variable with the given string name exists in the current environment.
`abs(number)`	Absolute value of an integer or float, preserving its type.
`pow(base, exponent)`	Exponentiation; see arithmetic.
`round(value[, digits])`	Round with halfway ties to even; digits default to zero. Negative digits round to tens, hundreds, etc. Return an integer when digits are zero or negative, otherwise a float. Nonfinite values and unsupported digit ranges are errors.
`min(value, ...)`, `max(value, ...)`	Select among one or more arguments. A single list argument is returned as a list, not reduced. Keep the first candidate on ties or incomparable comparisons.
`nan()`, `inf()`	Return NaN or positive infinity as a float.
`dice(count, sides)`	Sum count random integers from 1 through sides, inclusive. Count must be nonnegative and sides positive; arguments use integer conversion. Zero dice return zero.
`random()`	Random float in the inclusive interval from 0 to 1.

`2d6` is shorthand for `dice(2, 6)`. Both numbers must be decimal integer literals, separated by lowercase `d` without spaces. Use `dice(count, sides)` for computed arguments.

Random operations share one stream of draws. Reproducing results requires the same initial seed and the same sequence of operations and arguments. Random draws advance that stream; repeating a formula continues the sequence rather than restarting it. Short-circuiting, conditionals, and callbacks can change which random operations run. The language has no seed-setting expression.

1.7.2 Collections and callbacks

Call	Behavior
`len(value)`	Length of a list, dictionary, or string.
`range(stop)`	List of integers from zero up to, excluding, stop.
`range(start, stop[, step])`	Integer sequence with exclusive stop. Step defaults to 1, may be negative, and cannot be zero. Arguments use integer conversion.
`zip(list, ...)`	Rows of corresponding elements, stopping at the shortest list. Supply at least one list.
`fromentries(pairs)`	Build a dictionary from two-element lists `[string_key, value]`; later duplicates win.
`indexof(container, value)`	First matching list index, string substring index in Unicode scalar values, or dictionary key whose value matches. Return `null` when absent.
`sum(list)`, `mul(list)`	Numeric sum or product. Booleans count as 0/1; a float makes the result floating-point. Empty sum is 0; empty product is 1.
`enum(list)`, `flatten(list)`	Equivalent to the corresponding list methods.
`map(list, f)`, `filter(list, f)`	Equivalent to list `.map(f)` and `.filter(f)`.
`drop(list, count)`	Equivalent to list `.drop(count)`: remove the last count elements.

`cumul(list, f[, initial])`, `reduce(list, f[, initial])`	Equivalent to the corresponding list accumulator methods.
`apply(f, arguments)`	Call f with the list's elements as positional arguments; equivalent to `arguments.apply(f)`.
`method(value, name)`	Look up a method on a value and return it if found, `null` otherwise.
`order(a, b)`	Compare any two values using total ordering. Return −1, 0, or 1 when the first is less than, equal in order to, or greater than the second.

`order` uses a total ordering between all possible values. Incompatible types are sorted by their types in this order:

```
null < int, float and bool < string < list < dict < function
```

Booleans count as `0` and `1`. Numeric total ordering distinguishes negative zero from positive zero and places negative NaNs before negative infinity and positive NaNs after positive infinity. Mixed integer/float ordering preserves integer precision.

Lists compare length first, then compare corresponding elements by total ordering when lengths match. Dictionaries compare their entries in sorted-key order: each key is compared first, then its value; a shared entry prefix is followed by a length comparison. All functions tie in this ordering, even when they are unequal under == or ===.

1.7.3 Evaluation and errors

`eval(value[, bindings])` returns non-string values unchanged. For a string recognized as one complete expression, optionally wrapped in `${...}`, it evaluates that expression and preserves its result type. Other strings are processed as formatted text. The optional dictionary supplies local bindings that override the calling environment. Thus `eval('1 + 2')` produces 3, and `eval('${x + 1}', {'x': 2})` also produces 3.

An unwrapped expression may have surrounding whitespace. A `${...}` wrapper must occupy the entire string to preserve the result type; surrounding whitespace or text instead selects formatting and returns a string.

`error(message)` stops evaluation with a user error containing the message's text representation. `error()` uses the default message `Error!`. Additional arguments are evaluated but ignored.

`try(f[, if_err])` attempts to call the function `f`. If it succeeds, its result is returned. If it fails, the optional `if_err` function is called with the error dictionary; if `if_err` is not provided, `null` is returned.

1.8 Type methods

The signatures in this section omit the receiver. For example, `.len()` means `value.len()`. Booleans, null, and functions have no registered type methods.

1.8.1 Integer and float methods

Method	Behavior
`.abs()`, `.pow(exponent)`	Absolute value and exponentiation, as with the globals. Available on integers and floats.
`.min(value, ...)`, `.max(value, ...)`	Global min/max with the receiver as the first candidate. No additional argument is required.
`.clamp(low, high)`	Limit the value to inclusive bounds. Integer receivers convert bounds to integers and require low not to exceed high. Float receivers convert bounds to floats, swap reversed bounds, and reject NaN bounds.
`.sign()`	Integer result indicating sign. Integers return −1, 0, or 1. Floats return −1 or 1 according to the sign bit, including signed zero; NaN gives 0.

`.lerp(start, end)`	Use the receiver as the factor: `start + (end - start) * receiver`. Return a float. The factor is not clamped.
Integer: `.divmod(divisor)`	Return `[quotient, remainder]` with the quotient rounded down; a nonzero remainder has the divisor's sign. The divisor uses integer conversion. Reject zero divisors and quotient overflow.
Integer: `.tdivmod(divisor)`	Return `[quotient, remainder]` with the quotient rounded toward zero; a nonzero remainder has the dividend's sign. The divisor uses integer conversion. Reject zero divisors and quotient overflow.
Integer: `.rotl(count)`, `.rotr(count)`	Rotate the 64-bit representation left or right. Count must be a nonnegative integer; the effective count is modulo 64.
Float: `.round([digits])`	Same rounding and return types as global `round`.
Float: `.ceil()`, `.floor()`	Round upward or downward and convert to an integer.
Float: `.bits()`	Return the raw floating-point bit pattern as a signed 64-bit integer.
Float: `.isnan()`, `.isinf()`	Boolean tests for NaN or either infinity.

Parentheses make numeric receivers clear: `(-3).abs()` and `(2.5).round()`. For float-to-integer results, the saturation and NaN conversion rules also apply to `.ceil()` and `.floor()`.

1.8.2 String methods

Method	Behavior
`.len()`	Number of Unicode scalar values.
`.indexof(text)`	First substring's index in Unicode scalar values, or `null`.
`.lower()`, `.upper()`	Unicode lowercase or uppercase conversion.
`.trim()`	Remove leading and trailing whitespace.
`.replace(old, new)`	Replace all occurrences of old with new. Both arguments are strings.
`.split(separator)`	Split by a literal string separator; retain empty pieces. An empty separator includes empty leading/trailing pieces and one piece per Unicode scalar value.
`.startswith(prefix)`, `.endswith(suffix)`	Boolean prefix or suffix test.
`.capitalize()`	Uppercase the first character; leave the rest unchanged.
`.title()`	Capitalize each whitespace-separated word and join with single spaces; other letters retain their case.
`.slug()`	Lowercase, then replace every non-ASCII-alphanumeric character with _. Runs are not collapsed.
`.zfill(width[, fill])`	Left-pad to a character width. Fill defaults to `'0'` and must be one character; width must be nonnegative. Padding precedes any sign.
`.slice(start, end)`	Substring with exclusive end. Negative bounds count from the end; normalized bounds must be within the string and start must not exceed end. No step argument.
`.fmt([bindings])`	Format substitutions into a string using an optional dictionary of local bindings. Extra arguments are evaluated but ignored.

String `.slice` validates bounds rather than clamping them like bracket slicing. Its arguments use integer conversion, while bracket bounds require integers or `null`.

1.8.3 List methods

1.8.3.1 Values, ordering, and selection

Method	Behavior
`.len()`	Number of elements.
`.sum()`, `.mul()`	Numeric sum/product, as with the globals.
`.min()`, `.max()`	Select an element from a nonempty list. Incomparable comparisons are treated as ties; min keeps the first tied element and max the last.
`.indexof(value)`	First equal element's index, or `null`.
`.unique()`	Keep the first occurrence of each value according to equality. Nested lists and dictionaries are supported; elements need not be hashable.
`.enum()`	Produce `[index, value]` pairs, starting at index zero.
`.flatten()`	Remove one level of nested lists; retain non-list items.
`.flat()`	Recursively flatten all nested lists.
`.sort()`	Stable ascending sort using the total ordering described under `order`.
`.reverse()`	Return elements in reverse order.
`.take(count)`	First count elements; count must be nonnegative.
`.skip(count)`, `.drop(count)`	Discard the first or last count elements; count must be nonnegative.
`.slice(start, end)`	Exclusive-end slice. Start must be nonnegative; a negative end counts from the end. End must not precede start; bounds beyond the list are effectively truncated.
`.chunks(size)`	Non-overlapping groups; the last may be shorter. Size must be positive.
`.windows(size)`	All overlapping groups of exactly size elements. Size must be positive.
`.tryget(index[, default])`	Indexed value, or default when out of bounds. Negative indexes count from the end. Default is `null`.
`.pop([index])`	Return `[remaining_list, removed_value]`. Index defaults to the last element and supports negative indexing. Empty lists and invalid indexes fail.
`.coalesce()`	First non-null value, or `null`. Falsey non-null values are retained. All list elements have already been evaluated; this method does not short-circuit their evaluation like `??`.
`.join(separator)`	Join display representations with a string separator.
`.fmt()`	Concatenate display representations with no separator. Function elements are first called with no arguments.
`.choice()`	Uniformly select one element; the list must be nonempty.
`.weighted()`	Select from `[value, weight]` pairs. Use finite nonnegative weights with a positive finite total.
`.shuffle()`	Return a shuffled copy.

Count/index/size arguments in these methods must be integers. List `.slice` has different bound rules from bracket slicing and does not accept a step.

`.sort()` orders by the same logic as the `order` function described above.

1.8.3.2 Callbacks

List callbacks receive `(value, index)`, with zero-based indexes, unless otherwise specified. A user function may declare only the arguments it needs.

Method	Behavior
`.map(f)`	Apply f to each item and return the results.
`.filter(f)`	Keep items whose callback result is truthy.
`.find(f[, default])`	First item with a truthy callback result, or default (`null` when omitted). Stop at the first match.
`.all([f])`, `.any([f])`	Test item truthiness, or callback truthiness if supplied. Short-circuit at the first decisive result. Empty lists give true for all and false for any.
`.sortby(f)`	Stable ascending sort by computed keys. Each key is evaluated once. Keys must support ordinary comparisons.
`.minby(f)`, `.maxby(f)`	Return an item with the smallest/largest computed key under total ordering. Keep the first tie; empty lists fail.
`.extremeby(f)`	Return `[minimum_item, maximum_item]` by computed key under total ordering. Keep the first tie for each extreme; empty lists fail.
`.groupby(f)`	Dictionary of lists, grouped by string keys returned by f. Preserve first-seen group order and original item order.
`.cumul(f[, initial])`	Return successive accumulator values. Callback receives `(accumulator, value, index)`.
`.reduce(f[, initial])`	Return the final accumulator. Callback receives `(accumulator, value, index)`.
`.apply(f)`	Call f once, passing all list elements as positional arguments.

For `sortby`, `minby`, `maxby`, `extremeby`, and `groupby`, each key function is called exactly once per element, in input order, before sorting, selection, or grouping. `sortby` rejects incomparable keys, including a key that cannot compare with itself in a one-element list; the extreme-selection methods use total ordering instead.

Without an initial accumulator, `cumul` and `reduce` start with the first element and invoke the callback from index 1. `cumul` includes that first element in its output. With an initial value, both start processing at index 0; `cumul` includes only the updated accumulators, not the initial value itself. Empty `cumul` returns `[]`; empty `reduce` requires an initial value and returns it. An explicit `null` is an initial value, not an omitted argument: `[].reduce(fn total, value -> value, null)` returns `null`.

```
[2, 4].map(fn value, index -> value + index)
[1, 2, 3].cumul(fn total, value -> total + value)
[1, 2, 3].reduce(fn total, value -> total + value, 10)
[2, 3].apply(fn a, b -> a * b)
```

The results are `[2, 5]`, `[1, 3, 6]`, `16`, and `6`. Defaults passed to `find` and `tryget` are evaluated before the method runs, even when not needed.

1.8.4 Dictionary methods

Method	Behavior
`.len()`	Number of entries.

`.keys()`, `.values()`, `.entries()`	Lists in insertion order; entries are `[key, value]` pairs.
`.indexof(value)`	First key whose value compares equal, or `null`.
`.tryget(key[, default])`	Stored value for a string key, or default (`null` when omitted). A stored null value is returned as null.
`.remove(key)`	Copy without that string key. An absent key leaves contents unchanged.
`.haspath(path)`	Whether the dotted path exists through nested dictionaries/lists.
`.getpath(path[, default])`	Value at the dotted path. Missing keys/indexes use the supplied default or raise an error. Invalid traversal types still fail.
`.map(f)`	Map values while preserving keys and order. Callback receives `(value, key)`.
`.filter(f)`	Keep entries with truthy callback results. Callback receives `(value, key)`.

Paths are strings such as `'people.0.name'`. Segments that parse as nonnegative indexes address lists; other segments address dictionary keys. Numeric dictionary keys and keys containing dots cannot be addressed through this path syntax; use direct indexing instead. Path traversal uses stored data, not registered methods.

1.9 Text formatting and embedded formulas

Formatting scans a string for substitutions:

- `$name` inserts the display representation of a visible variable. An unknown shorthand name remains unchanged in the text.
- `${expression}` evaluates an expression and inserts its display representation. Nested dictionary braces and quoted strings are handled inside the delimiters. Evaluation errors propagate.
- `$$` inserts one literal dollar sign and prevents that sign from starting a substitution.

Use a string's `.fmt()` method to format explicitly. Its optional dictionary adds or overrides bindings. Formatting always returns a string, even if the whole input is one substitution.

```
'Hello, $name!'.fmt({'name': 'Ada'})
'Total: ${price * quantity}'.fmt({'price': 4, 'quantity': 3})
'Cost: $$5'.fmt()
eval('${1 + 2}')
'${1 + 2}'.fmt()
```

These return `'Hello, Ada!'`, `'Total: 12'`, `'Cost: $5'`, the integer `3`, and the string `'3'`. Formatting is a single pass: text produced by a substitution is not scanned again. A `${` without a matching closing brace remains literal text.